COMPUTING 2

Parallel Computing Midterm Study Guide

Explanatory edition with readable equations, visual tables, and integrated practice

GUIDE AT A GLANCE

How to read: Use this table to see what changed from the compact edition.

Design choice	What it does for studying
Narrative before formulas	Explains the question each equation answers before introducing symbols
Large equation cards	Separates the formula, variable meaning, and interpretation
Titled tables	Makes the comparison goal and reading direction explicit
Integrated practice	Places each problem and its full solution beside the related concept
Visual execution models	Shows hierarchy, coalescing, false sharing, and Roofline behavior

Recommended study loop
Read one section slowly. Cover the green solution box and solve the coral problem. Then compare your reasoning, not only the final number. Finish by explaining the equation in one sentence without looking.

Lecture snapshot: GitHub main commit 3481aa702f262f138bf4c8db6fb068b4e5d316d7
Prepared: 2026-10-17

Contents

Contents 2

0. How the Course Fits Together 4

1. Performance Models: What Limits Speedup? 5

1.1 Start with time, not processor count . . . 5

1.2 Amdahl's law: the serial remainder dominates . . . 6

1.3 Gustafson's law: scale the useful work . . . 7

1.4 Roofline: connect computation to memory traffic . . . 8

2. Designing a Parallel Algorithm 11

2.1 Find independence before choosing an API . . . 11

2.2 Scheduling, locality, and communication . . . 11

3. CPU Parallelism: ILP, SIMD, SMT, and Multicore 13

3.1 Different mechanisms parallelize different things . . . 13

3.2 Superscalar and out-of-order execution . . . 13

3.3 SIMD/vector execution . . . 13

3.4 SMT, multicore, and resource sharing . . . 14

3.5 Coherence, consistency, and false sharing . . . 14

4. GPU Architecture: Throughput by Massive Threading 15

4.1 CPU and GPU design goals . . . 15

4.2 Grid, block, warp, and thread . . . 15

4.3 NVIDIA, AMD GCN, and Arm Valhall . . . 16

4.4 Divergence: correct result, lower lane utilization . . . 16

4.5 Memory spaces: scope and reuse determine the choice . . . 17

4.6 Coalescing: inspect addresses across lanes . . . 17

5. CUDA Programming: Map Work, Synchronize, Reuse Data 19

5.1 Host-device flow and one-dimensional indexing . . . 19

5.2 Two-dimensional indexing . . . 20

5.3 The transpose race: identify the producer . . . 21

5.4 Why tiling reduces matrix-multiplication loads . . . 22

5.5 Tiled GEMM as repeated load-compute phases . . . 23

5.6 Occupancy: the minimum of several resource limits . . . 24

6. OpenMP, MPI, and Hybrid Programming 25

6.1 Choose a model that matches the memory system . . . 25

6.2 OpenMP fork-join and data sharing . . . 25

6.3 Scheduling and tasks . . . 25

6.4 MPI collectives and row-partitioned matvec . . . 26

6.5 Blocking-message deadlock and hybrid design . . . 26

7. Exam Connections and Final Review 27

7.1 How the practice-midterm questions map to concepts 27

7.2 Formula sheet with interpretation 27

7.3 Common reasoning mistakes 27

8. References 29

0. How the Course Fits Together

Parallel computing is not one trick. It is a chain of decisions. First decide what work can happen together. Then choose the hardware level that can execute it. Finally map work and data so that communication, synchronization, and memory traffic do not erase the benefit of parallel execution.

THE COURSE AS ONE REASONING CHAIN

How to read: Move from left to right when solving a design or code-analysis question.

Question	Main idea	Typical exam evidence
How much speedup is even possible?	Amdahl, Gustafson, Roofline	Derive a bound and name the bottleneck
What can execute together?	Dependencies, decomposition, granularity	Identify independent work and overhead
Where does the parallelism live?	ILP, SIMD, SMT, cores, GPU warps	Compare execution and shared resources
How are indices and data mapped?	CUDA hierarchy, coalescing, tiling	Derive an index or count memory requests
How is correctness preserved?	Barriers, races, reductions, collectives	Find missing synchronization or communication

Three questions to ask everywhere

1. What is independent? 2. What data must move or be shared? 3. What overhead or hardware limit prevents ideal scaling?

1. Performance Models: What Limits Speedup?

1.1 Start with time, not processor count

A parallel machine is useful only if elapsed time decreases. Processor count by itself says nothing about success. The serial baseline T_1 is therefore the reference point, and the measured parallel time T_p includes useful computation plus every parallel cost: communication, synchronization, scheduling, and load imbalance.

SPEEDUP

$$S(p) = T_1 / T_p$$

S=4 means the parallel execution finishes in one quarter of the serial time.

EFFICIENCY

$$E(p) = S(p) / p$$

Efficiency asks how much of ideal linear speedup each processor contributes on average.

PARALLEL OVERHEAD

$$T_o = p\,T_p - T_1$$

This converts all parallel resource time into one number and subtracts the useful serial work.

WORKED EXAMPLE - Measured scaling

Suppose T_1=80 s and T_8=14 s. Then S=80/14=5.71 and E=5.71/8=71.4%. The missing 28.6% is not one specific cost; it summarizes serial work, imbalance, communication, and other overheads.

TRY IT NOW - Speedup and efficiency
A program takes 120 s serially and 22 s on 8 processors. Find speedup and efficiency. Is the result closer to ideal scaling or to no scaling?

SOLUTION
S=120/22=5.455. E=5.455/8=0.682, or 68.2%. It is substantially faster than serial execution, but ideal 8x scaling would require 15 s, so parallel costs remain significant.

ANSWER EQUATION

$$S=5.455,\ E=68.2\%$$

The full reasoning is shown directly above; no answer-key lookup is needed.

1.2 Amdahl's law: the serial remainder dominates

Amdahl's law answers a fixed-size question: if the total amount of work stays the same, how fast can p processors finish it? Normalize the serial time to 1. Let s be the fraction that cannot be parallelized. That fraction still takes s time. The remaining fraction 1-s is divided ideally across p processors, so it takes (1-s)/p time.

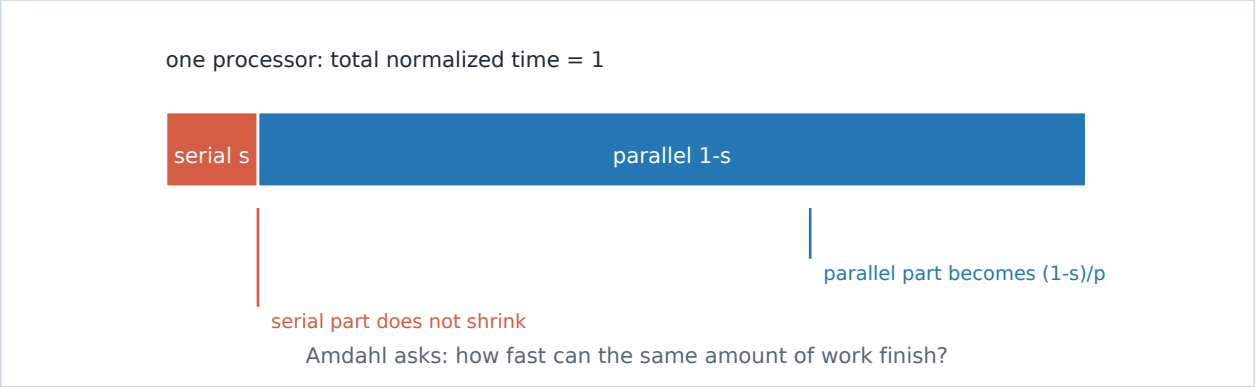


Figure 1. The derivation comes directly from splitting normalized serial time into a non-shrinking part and a divisible part.

AMDAHL FIXED-SIZE SPEEDUP

$$S_A(p) = 1 / [\ s + (1-s)/p\ ]$$

The denominator is the remaining fraction of serial runtime after ideal parallelization.

AMDAHL CEILING

$$\lim_{p \to \infty} S_A(p) = 1/s$$

Even infinitely many processors cannot remove the serial fraction.

WORKED EXAMPLE - The practice-midterm numbers
With s=0.10 and p=8, the normalized parallel time is 0.10+0.90/8=0.2125. Therefore S=1/0.2125=4.706. With infinitely many processors, the 0.90/p term vanishes and the maximum speedup is 1/0.10=10.

TRY IT NOW - Infer the serial fraction
A fixed-size program reaches speedup 5 on 8 processors. Under ideal Amdahl assumptions, estimate s and the maximum speedup.

SOLUTION
Start from 1/5=s+(1-s)/8=1/8+7s/8. Thus s=(0.2-0.125)/(7/8)=0.085714. The ceiling is 1/s=11.667. This inverse form is useful because measured speedup can reveal an effective serial fraction.

ANSWER EQUATION

$$s = (1/S - 1/p)/(1 - 1/p) = 0.085714$$

The full reasoning is shown directly above; no answer-key lookup is needed.

What Amdahl does not say
It does not claim that adding processors is useless. It says that for one fixed problem, returns diminish unless the serial fraction and overheads are reduced.

1.3 Gustafson's law: scale the useful work

Amdahl fixes the problem size. Gustafson asks a different question: if the parallel run is allowed to solve a larger problem in the same elapsed time, how much serial-equivalent work was completed? Normalize the parallel elapsed time to 1. The serial part costs s, while the scalable part would take p(1-s) on one processor.

GUSTAFSON SCALED SPEEDUP

$$S_G(p) = s + p(1-s) = p - (p-1)s$$

The workload grows with processor count while elapsed time stays fixed.

AMDahl VS. GUSTAFSON: THE QUESTION CHANGES

How to read: The laws are not contradictory; the held-fixed quantity differs.

Model	Held fixed	Question	Main lesson
Amdahl	Problem size	How fast can the same job finish?	Serial work creates a strong ceiling
Gustafson	Elapsed time	How much larger a job fits in the same time?	Parallel machines can absorb scaled work

TRY IT NOW - Scaled speedup

A 32-processor run spends 5% of its elapsed time in serial work. Compute Gustafson's scaled speedup and state what grew.

SOLUTION

S_G=32-31(0.05)=30.45. The elapsed time did not shrink; instead, the parallel portion represents a larger problem than one processor could complete in that time.

ANSWER EQUATION

$$S_G = 30.45$$

The full reasoning is shown directly above; no answer-key lookup is needed.

1.4 Roofline: connect computation to memory traffic

Amdahl describes parallel fractions, but many kernels are limited by data movement. Roofline compares two independent ceilings. The compute ceiling says the machine cannot exceed peak arithmetic throughput. The bandwidth ceiling says that moving B bytes per second and performing I FLOPs per byte can supply at most B x I FLOPs per second.

OPERATIONAL INTENSITY

$$I = \text{number of FLOPs / bytes moved from the measured memory level}$$

Count useful arithmetic and the bytes crossing the chosen memory boundary.

ROOFLINE BOUND

$$P_{attainable} <= \min(P_{peak}, B\ I)$$

The lower of the compute roof and bandwidth roof is the active limit.

RIDGE POINT

$$I^* = P_{peak} / B$$

Left of I* is memory-bound; right of I* is compute-bound.

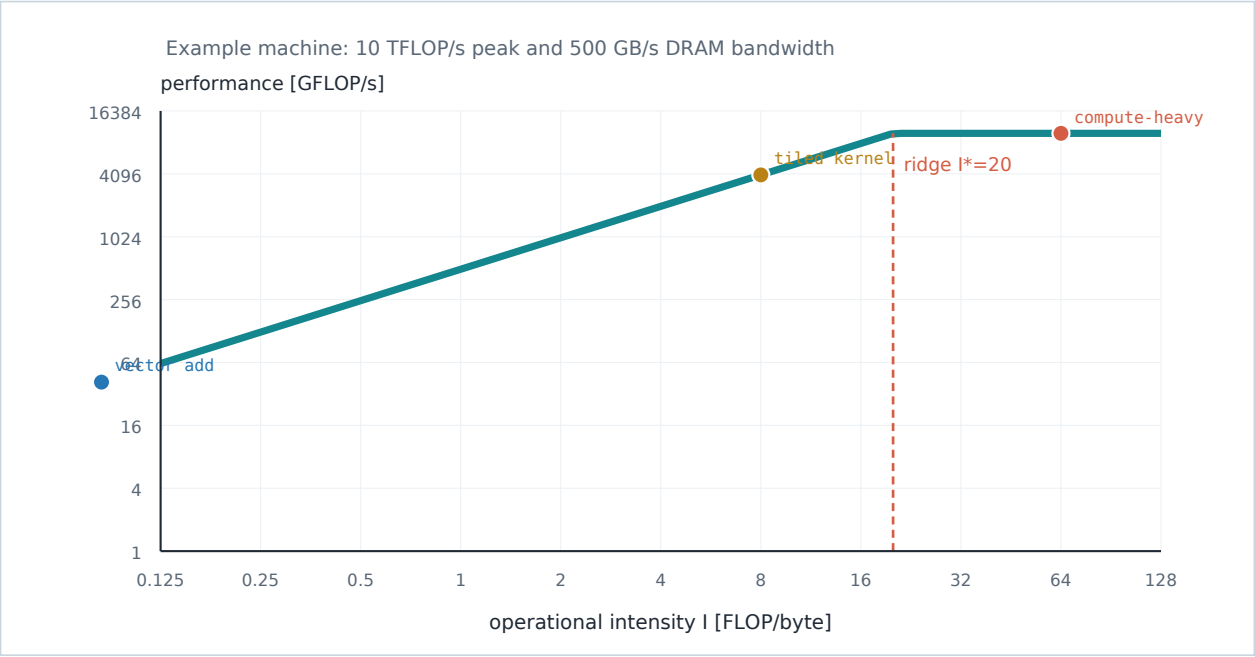


Figure 2. The sloped region is bandwidth-limited. The horizontal region is compute-limited.

WORKED EXAMPLE - Why vector addition is memory-bound

For C[i]=A[i]+B[i] with FP32 data, one element reads 8 bytes, writes 4 bytes, and performs one add. Ignoring write allocation, I=1/12=0.083 FLOP/byte. On a 900 GB/s machine, B I=75 GFLOP/s, far below a multi-TFLOP compute peak.

TRY IT NOW - Roofline classification

A machine has P_{peak}=12 TFLOP/s and B=600 GB/s. Find the ridge point and bound kernels with I=4 and I=30 FLOP/byte.

SOLUTION

Convert 12 TFLOP/s to 12,000 GFLOP/s. The ridge is 12,000/600=20 FLOP/byte. At I=4, BI=2,400 GFLOP/s=2.4 TFLOP/s, so the kernel is memory-bound. At I=30, BI=18 TFLOP/s exceeds peak, so the bound is 12 TFLOP/s and the kernel is compute-bound.

ANSWER EQUATION

$$I^*=20;\ P(4)<=2.4\ \text{TFLOP/s};\ P(30)<=12\ \text{TFLOP/s}$$

The full reasoning is shown directly above; no answer-key lookup is needed.

2. Designing a Parallel Algorithm

2.1 Find independence before choosing an API

Parallelization begins with a dependency question, not with OpenMP or CUDA syntax. Two tasks may overlap only if neither needs a value that the other has not produced. A correct dependency graph exposes available parallelism and the critical path. The critical path is the longest chain of required ordering and therefore a lower bound on execution time.

DECOMPOSITION CHECKLIST

How to read: Each row removes a different reason that parallel hardware may sit idle.

Check	Question to ask	Failure symptom
Dependencies	Which outputs feed later tasks?	Race or incorrect reordering
Granularity	Is each task large enough to amortize overhead?	Scheduling dominates useful work
Task count	Are there enough ready tasks for all resources?	Idle cores despite remaining work
Balance	Do tasks have similar cost or adaptive scheduling?	Some workers finish much earlier
Locality	Does reused data stay near the worker?	Excess cache misses or communication

Fine-grained tasks expose more parallelism but pay more scheduling and synchronization overhead. Coarse-grained tasks reduce overhead and may improve locality, but they can leave too few tasks or create imbalance. Good granularity lies between these extremes.

TRY IT NOW - Task granularity
A loop has 1,000,000 independent iterations, each taking only 20 ns, while scheduling one task costs 2 microseconds. Should each iteration be a task? Suggest a better decomposition.

SOLUTION
No. One task would cost about 100 times more to schedule than to compute. Group many iterations into a chunk so that useful work greatly exceeds 2 microseconds, while retaining enough chunks for balance. For example, 1,000 iterations per task gives roughly 20 microseconds of work per 2-microsecond scheduling cost.

2.2 Scheduling, locality, and communication

Static scheduling assigns work in advance and has low overhead, but assumes costs are predictable. Dynamic scheduling assigns work as workers finish and can repair imbalance, but every assignment costs runtime work and may disrupt locality. The correct choice depends on the variance of task cost, not on a universal preference.

COMMUNICATION COLLECTIVES BY DATA MOVEMENT

How to read: Identify who starts with data and who must own the result.

Collective	Before	After	Typical use
Broadcast	One root has x	Every rank has x	Replicate read-only input
Scatter	Root has all chunks	Each rank has one distinct chunk	Distribute rows or blocks
Gather	Each rank has one chunk	Root has concatenated result	Collect distributed output

Collective	Before	After	Typical use
All-gather	Each rank has one chunk	Every rank has all chunks	Replicate a distributed vector
Reduce	Ranks have partial values	Root has combined value	Sum, max, or other associative result
All-reduce	Ranks have partial values	Every rank has combined value	Global norm or synchronized gradient

TRY IT NOW - Distributed matrix-vector multiplication

Rows of A are distributed across MPI ranks, but x begins at rank 0. Which communication and local computation steps produce y at rank 0?

SOLUTION

Broadcast x to all ranks. Each rank multiplies its local rows by x, producing a local segment of y. Gather those segments at rank 0. If every rank needs the full y, replace Gather with Allgather.

Optimization order

First establish a correct serial baseline. Then measure hotspots. Parallelize the dominant work, verify correctness, and measure end-to-end time across sizes and processor counts. Optimizing a non-hotspot cannot create large total speedup.

3. CPU Parallelism: ILP, SIMD, SMT, and Multicore

3.1 Different mechanisms parallelize different things

The word parallelism hides several levels. Instruction-level parallelism (ILP) overlaps independent instructions from one thread. Data-level parallelism applies one instruction to multiple elements. Thread-level parallelism runs independent instruction streams. Process-level parallelism uses separate address spaces and explicit communication.

FOUR LEVELS OF PARALLELISM

How to read: Read the second column as the unit duplicated or overlapped.

Level	Parallel unit	Hardware/software example	Main constraint
ILP	Instructions from one thread	Superscalar out-of-order core	Dependencies and issue width
DLP	Data lanes/elements	SIMD or vector instruction	Regular independent operations
TLP	Threads	SMT or multicore	Synchronization and shared resources
Process	Processes/ranks	MPI across nodes	Communication and distributed data

3.2 Superscalar and out-of-order execution

A superscalar core can issue more than one instruction per cycle. Out-of-order logic tracks dependencies and chooses ready instructions from a window, even when an earlier instruction is waiting. Issue width is a ceiling, not a guarantee: true dependencies, branch mispredictions, cache misses, and unavailable functional units can leave issue slots empty.

TRY IT NOW - Why four-wide is not four instructions every cycle
A four-wide core executes a dependency chain a=b+c; d=a+e; f=d+g. Why can it not issue all three additions together?

SOLUTION
The second addition needs a from the first, and the third needs d from the second. These are read-after-write dependencies, so the three instructions form a serial chain even though the core could issue four independent instructions in one cycle.

3.3 SIMD/vector execution

A vector instruction describes the same operation on several elements. This increases data throughput and reduces instruction fetch/decode demand because one instruction replaces many scalar arithmetic instructions. It works best when iterations are independent and memory accesses are regular.

WORKED EXAMPLE - Instruction-count intuition
For 1024 additions with vector length 8, the arithmetic operation requires 1024 scalar add instructions or 1024/8=128 vector add instructions. This does not promise an exact 8x runtime reduction because loads, stores, dependencies, and bandwidth still matter.

TRY IT NOW - Vector suitability
Which loop vectorizes more naturally: (A) y[i]=2*x[i]+1 or (B) x[i]=x[i-1]+1? Explain.

SOLUTION
Loop A has independent iterations and regular access. Loop B has a loop-carried dependency because iteration i needs the result from i-1, so straightforward SIMD execution would violate the required order.

3.4 SMT, multicore, and resource sharing

SMT VS. MULTICORE VS. SIMD

How to read: The mechanisms can coexist; compare what is shared and what runs independently.

Mechanism	Independent state	Shared resource	Why it helps
SMT	Multiple hardware thread contexts	One core's pipelines and caches	Uses issue slots during another thread's stalls
Multicore	Separate cores and thread contexts	Often last-level cache and memory bandwidth	Provides independent execution engines
SIMD/vector	Multiple data lanes	One instruction/control stream	Amortizes instruction control across elements

SMT helps most when threads have complementary stalls and resource demands. It can hurt when both threads compete for an already saturated front end, execution port, cache, or memory channel.

3.5 Coherence, consistency, and false sharing

Cache coherence is mainly a per-address promise: writes to one location become visible in an agreed order. Memory consistency is the broader software-visible contract that constrains ordering across operations and addresses. Synchronization creates the ordering relationships required by a correct program.

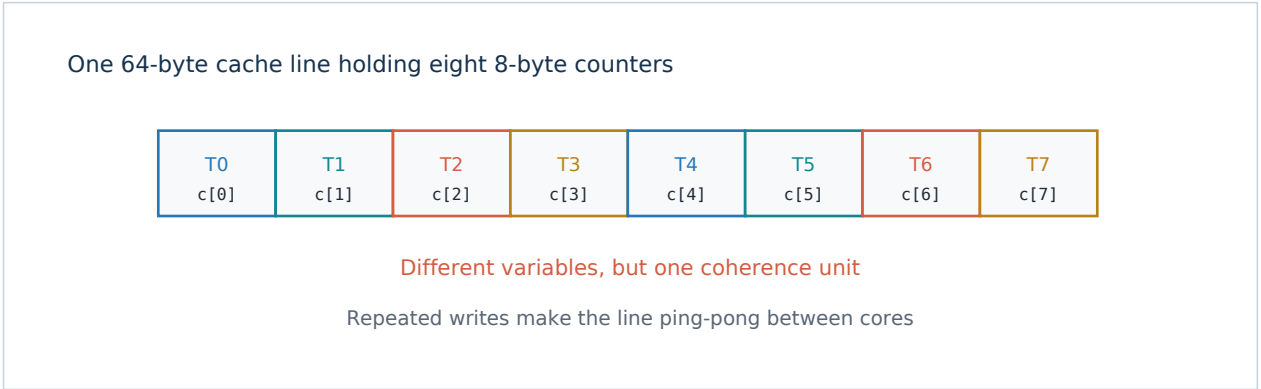


Figure 3. False sharing occurs because coherence tracks cache lines, not source-language variables.

When eight threads repeatedly update eight adjacent 8-byte counters, all counters may occupy one 64-byte cache line. No logical variable is shared, yet each write invalidates the line in other caches. Padding each counter to a line or accumulating privately and writing once removes the ping-pong.

TRY IT NOW - False sharing repair
Each OpenMP thread repeatedly increments partial[tid]. The values are adjacent doubles. Give the lowest-overhead repair when only the final totals matter.

SOLUTION
Keep the running value in a thread-private scalar, then write partial[tid] once after the loop or use a reduction. Padding also works, but private accumulation avoids frequent shared-cache-line writes entirely.

4. GPU Architecture: Throughput by Massive Threading

4.1 CPU and GPU design goals

A CPU spends substantial area and power reducing the latency of a few instruction streams: deep caches, branch prediction, and aggressive out-of-order scheduling. A GPU spends more resources on arithmetic lanes and thread state so that many warps can be active. When one warp waits for memory, the SM issues another ready warp. This is latency hiding, not low memory latency.

LATENCY-ORIENTED CPU VS. THROUGHPUT-ORIENTED GPU

How to read: The comparison is about design emphasis, not an absolute rule.

CPU emphasis	GPU emphasis
Strong single-thread latency	High aggregate throughput
Complex control and large caches per core	Many simpler lanes and many resident threads
A few heavyweight threads	Thousands of logical threads
Good for irregular control and low-latency tasks	Good for regular data-parallel work

4.2 Grid, block, warp, and thread

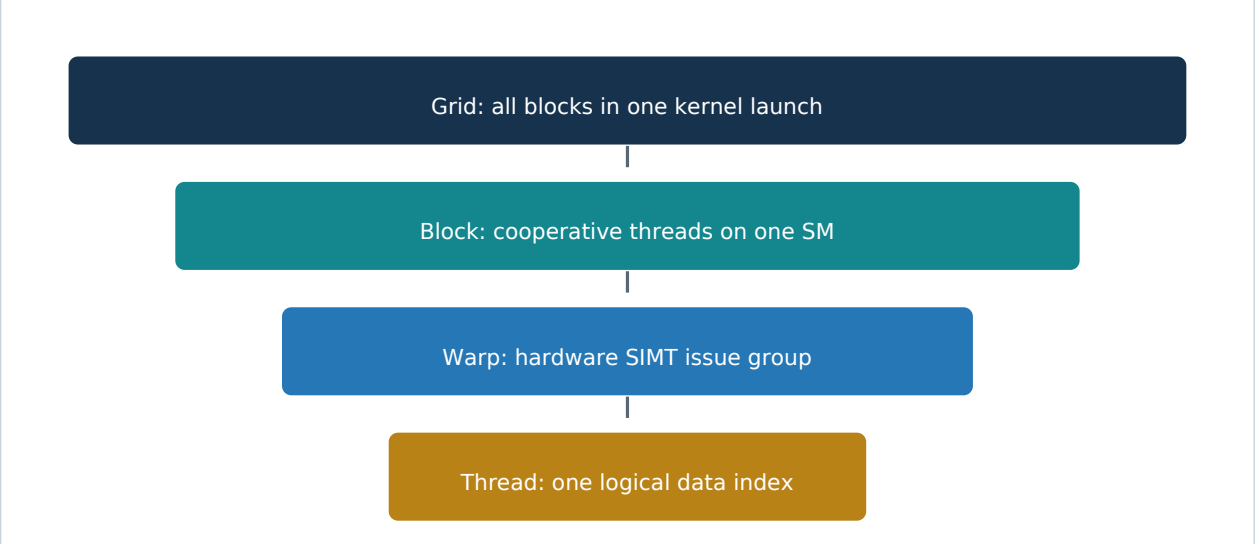


Figure 4. Software chooses grid and block dimensions. Hardware groups block threads into architecture-defined warps.

A block is the cooperation boundary. Its threads reside on one SM, share the block's shared memory, and may use __syncthreads(). A warp is the hardware SIMT issue group; in the course's NVIDIA context it contains 32 threads. A grid contains all blocks launched by one kernel, but ordinary CUDA provides no general grid-wide barrier inside a kernel.

TRY IT NOW - Launch accounting
A kernel launches 1000 blocks with 512 threads each. With 32-thread warps, count logical threads, warps per block, total logical warps, and lifetime instances of one scalar shared variable.

SOLUTION
Threads=1000x512=512,000. Warps/block=512/32=16. Total logical warps=16,000. Shared memory is per block, so the variable has 1000 logical instances over the launch, even though physical storage is reused as blocks come and go.

ANSWER EQUATION

512,000 threads; 16 warps/block; 16,000 warps; 1000 shared instances

The full reasoning is shown directly above; no answer-key lookup is needed.

4.3 NVIDIA, AMD GCN, and Arm Valhall

GPU EXECUTION-GROUP TERMINOLOGY

How to read: Widths and scheduling details are architecture-specific; use the assumptions in the question.

Family	Execution group	Course value	Scheduling description
NVIDIA CUDA	Warp	32 threads	SM warp scheduler issues ready warps
Historical AMD GCN	Wavefront	64 work-items	Wavefronts of a workgroup reside on one CU
Arm Valhall	Warp-like group	Implementation-dependent	Hardware dynamic instruction scheduling and superscalar issue

Exam trap
Software controls the block or workgroup size, but not the hardware warp or wavefront width. Therefore the statement 'warp size is controlled by software' is false.

4.4 Divergence: correct result, lower lane utilization

SIMT lanes in one warp share an instruction stream. If lanes choose different branches, the warp executes the paths in separate steps using active masks. The result remains correct, but lanes not on the current path sit idle. If all lanes choose the same path, the branch is uniform and there is no divergence.

WORKED EXAMPLE - A numeric divergence model
Suppose 16 lanes take a 20-cycle path and 16 take an 8-cycle path. The warp spends 28 cycles executing both. Useful lane-cycles are 16x20+16x8=448, while available lane-cycles are 32x28=896. Lane utilization is 50%.

TRY IT NOW - Uniform versus divergent branch
Compare if (threadIdx.x < 16) with if (blockIdx.x < 16) for a 32-thread warp. Which is more likely to diverge?

SOLUTION
threadIdx.x<16 splits one warp into two lane groups, so it diverges. blockIdx.x is identical for every thread in a block and therefore identical within a warp, so that branch is uniform for the warp.

4.5 Memory spaces: scope and reuse determine the choice

GPU MEMORY SPACES BY VISIBILITY AND PURPOSE

How to read: Choose memory by who reuses the value, not only by nominal latency.

Space	Visible to	Lifetime	Best use	Main caution
Registers	One thread	Thread	Accumulators and thread-local values	Pressure can reduce occupancy or spill
Shared memory	One block	Block	Cross-thread reuse, tiles, rearrangement	Needs synchronization; bank conflicts
Constant memory	All threads read-only	Allocation	Small uniform read-only data	Divergent addresses reduce benefit
Texture/read-only path	All threads read-only	Allocation	Spatially local read-only data	Benefit depends on access pattern
Global memory	Grid/device	Allocation	Large arrays	High latency and transaction efficiency
Unified Memory	CPU and GPU addressability	Allocation	Convenient shared pointer	Page migration and first-touch cost

TRY IT NOW - Shared memory or registers?
Capacity is not a problem. Why can shared memory still be necessary instead of registers for a transpose tile?

SOLUTION
Registers are private to one thread, so another thread cannot directly read them. A transpose requires threads to exchange or rearrange values. Shared memory provides block-visible indexed storage; threads load cooperatively, synchronize, and then read one another's staged values.

4.6 Coalescing: inspect addresses across lanes

A GPU memory instruction is issued for a warp. Transaction efficiency therefore depends on the set of addresses generated by neighboring lanes for that instruction. A thread reading two adjacent values over two instructions does not guarantee that either instruction is coalesced across the warp.

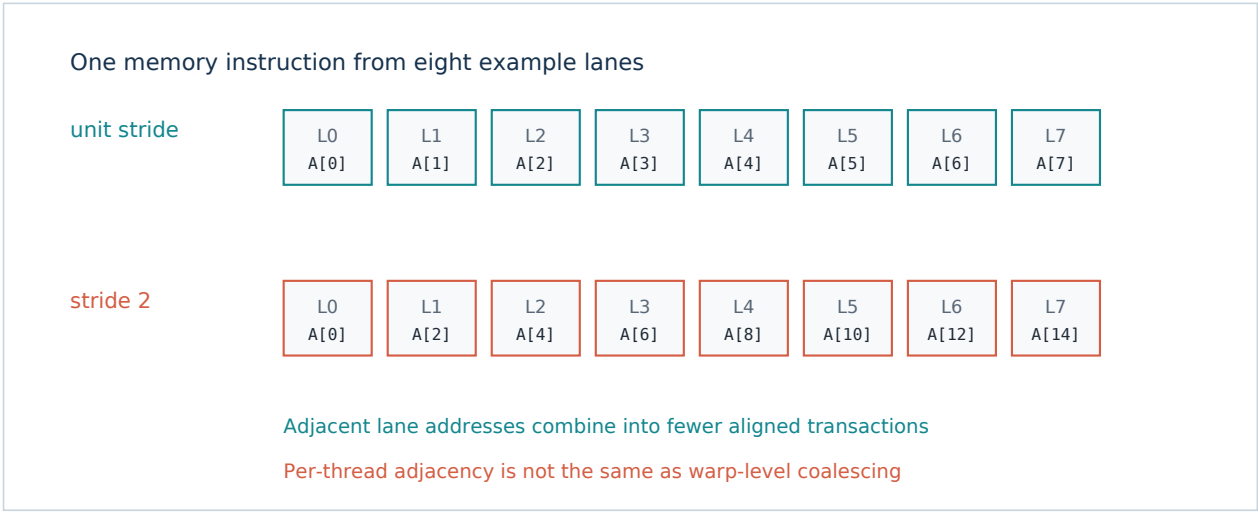


Figure 5. Unit stride uses contiguous addresses across lanes; stride 2 spans more memory segments.

TRY IT NOW - Count simplified transaction segments

A 32-thread warp reads FP32 data from a 128-byte aligned base. Under a simplified 128-byte segment model, compare A[t] and A[2t].

SOLUTION

A[t] touches byte offsets 0..127 and fits one segment with 100% useful bytes. A[2t] touches offsets 0..251, spanning two segments while using only half of the transferred words. It therefore uses two segments and 50% useful-byte efficiency.

5. CUDA Programming: Map Work, Synchronize, Reuse Data

5.1 Host-device flow and one-dimensional indexing

A CUDA program allocates memory, transfers or makes data accessible, launches a kernel, checks errors, synchronizes when results are needed, and retrieves results. Inside the kernel, the global one-dimensional thread index is the number of threads in earlier blocks plus the local thread offset.

ONE-DIMENSIONAL GLOBAL INDEX

$$i = blockIdx.x \times blockDim.x + threadIdx.x$$

blockIdx.x blockDim.x counts all threads in earlier blocks; threadIdx.x selects the lane within this block.

CEILING LAUNCH SIZE

$$gridDim.x = (n + blockDim.x - 1) / blockDim.x$$

Integer ceiling launches enough threads; the kernel must still guard i<n.

```
int i = blockIdx.x * blockDim.x + threadIdx.x;
if (i < n) C[i] = A[i] + B[i];
```

TRY IT NOW - Two coalesced sections per block

Each block processes two consecutive sections of blockDim.x elements. Every thread first handles one element from section 1, then one from section 2. Derive both indices.

SOLUTION

The block owns 2B elements where B=blockDim.x. Its base is blockIdx.x x 2B. Lane t first uses base+t, so neighboring lanes are contiguous. The second index is exactly one section later: base+B+t.

ANSWER EQUATION

$$i = blockIdx.x\ (2\ blockDim.x) + threadIdx.x;\ j = i + blockDim.x$$

The full reasoning is shown directly above; no answer-key lookup is needed.

```
int i = blockIdx.x * (2 * blockDim.x) + threadIdx.x;
int j = i + blockDim.x;
if (i < n) C[i] = A[i] + B[i];
if (j < n) C[j] = A[j] + B[j];
```

Why 2 x global-thread-id is less coalesced

i=2(blockIdx.x blockDim.x+threadIdx.x) gives first addresses 0,2,4,... across a warp. Each thread's pair may be adjacent, but the first warp instruction has stride 2. Coalescing is evaluated across lanes per instruction.

5.2 Two-dimensional indexing

For a row-major matrix, x naturally maps to columns and y to rows. The linear offset is row times the row width plus column.

2D ROW-MAJOR MAPPING

$$col=blockIdx.x\ blockDim.x+threadIdx.x;\ row=blockIdx.y\ blockDim.y+threadIdx.y;\ idx=row\ W+col$$

Guard both row<H and col<W before using idx.

TRY IT NOW - 2D bounds
A 1000 by 750 image uses 16 by 16 blocks. Give the grid dimensions and the bounds condition.

SOLUTION
grid.x=ceil(1000/16)=63 and grid.y=ceil(750/16)=47. The kernel uses if (col<1000 && row<750). Extra threads in the right and bottom edge blocks exit without accessing memory.

ANSWER EQUATION

grid=(63,47); guard col<1000 and row<750

The full reasoning is shown directly above; no answer-key lookup is needed.

5.3 The transpose race: identify the producer

In a shared-memory transpose, thread (x,y) writes tile[y][x] but reads tile[x][y]. Except on the diagonal, the reader depends on another thread's write. The crucial question is therefore: can every producer finish before any consumer begins? A block barrier between the store and load phases creates that ordering.

```
__shared__ float tile[T][T];
tile[threadIdx.y][threadIdx.x] = A[idx];
__syncthreads();
A[idx] = tile[threadIdx.x][threadIdx.y];
```

TRY IT NOW - Exact practice-midterm transpose condition
T ranges from 1 to 20 and the NVIDIA warp size is 32. Without the barrier, for which T does the intended lecture model treat the block as correct? How is it repaired for every T?

SOLUTION
The block has T^2 threads. It fits in one warp when $T^2<=32$, giving T=1,2,3,4,5. For T=6..20, multiple warps create a possible producer-consumer race. Insert __syncthreads() after all shared stores. In portable general code, use the barrier even if a one-warp case appears to work.

ANSWER EQUATION

T=1..5 in the course model; add __syncthreads() for T=1..20

The full reasoning is shown directly above; no answer-key lookup is needed.

Barrier safety rule
Every thread in the block must reach __syncthreads(). A barrier inside a branch taken by only some threads can deadlock or produce undefined behavior.

5.4 Why tiling reduces matrix-multiplication loads

For C[i,j], the naive algorithm loops over k. A[i,k] is reused for every output column j, and B[k,j] is reused for every output row i. Without cooperative staging, each consumer thread requests its own copy. Therefore each input element is requested N times, and the two input matrices generate $2N^3$ logical requests.

NAIVE GEMM REQUESTS

each input element: N requests; total: $2N^3$

N output positions reuse a particular input element, but naive threads request it independently.

TILED GEMM REQUESTS

each input element: N/T requests; total: $2N^3/T$

One shared-memory load is reused by T output threads, ideally reducing requests by T.

WORKED EXAMPLE - N=1024 and T=32
Naive: each input element is requested 1024 times. Tiled: 1024/32=32 times. The logical request reduction is 32x. Actual DRAM transactions also depend on caches, alignment, and write policy, but the practice question asks for algorithmic requests including caches.

TRY IT NOW - Practice-midterm request count
State the per-input-element request count for N by N GEMM without tiling and with T by T shared-memory tiling. Explain the division by T in one sentence.

SOLUTION
Without tiling: N requests per input element. With tiling: N/T requests. A staged value is loaded once for an output tile and reused by T threads along the reuse dimension.

ANSWER EQUATION

naive: N; tiled: N/T

The full reasoning is shown directly above; no answer-key lookup is needed.

5.5 Tiled GEMM as repeated load-compute phases

A tiled kernel repeats three actions: cooperatively load one A tile and one B tile, synchronize, then accumulate products from shared memory. A second barrier prevents a fast warp from overwriting the tile for the next phase while another warp is still using the current phase.

```
for (int ph = 0; ph < (N+T-1)/T; ++ph) {
  As[ty][tx] = valid_A ? A[...] : 0.0f;
  Bs[ty][tx] = valid_B ? B[...] : 0.0f;
  __syncthreads();
  for (int k=0; k<T; ++k) sum += As[ty][k] * Bs[k][tx];
  __syncthreads();
}
if (row<N && col<N) C[row*N+col] = sum;
```

IDEALIZED FP32 TILED INTENSITY

I approximately T/4 FLOP/byte

One phase performs 2T cubed FLOPs after loading 2T squared FP32 values = 8T squared bytes; output traffic is ignored.

TRY IT NOW - Tile size and operational intensity
Under the idealized formula, compare T=8 and T=32.

SOLUTION
At T=8, I approximately 2 FLOP/byte. At T=32, I approximately 8 FLOP/byte. Larger tiles expose more reuse, but tile size is limited by threads, shared memory, registers, bank behavior, and edge handling.

ANSWER EQUATION

$$I(8) \text{ approximately } 2;\ I(32) \text{ approximately } 8 \text{ FLOP/byte}$$

The full reasoning is shown directly above; no answer-key lookup is needed.

5.6 Occupancy: the minimum of several resource limits

Occupancy is the fraction of the SM's maximum threads or warps that are resident. A block consumes threads, registers, shared memory, and a block slot. Convert every resource into a block limit and take the minimum. High occupancy can hide latency, but it does not guarantee high performance if memory transactions, divergence, or instruction throughput dominate.

RESIDENT BLOCKS

$$\text{blocks/SM} = \min(\text{thread limit}, \text{register limit}, \text{shared-memory limit}, \text{block-count limit})$$

Each limit is a floor division of the SM resource by the block's demand.

THREAD OCCUPANCY

$$\text{occupancy} = \text{resident threads} / \text{maximum resident threads}$$

Use the resident block count to compute resident threads.

WORKED EXAMPLE - Three competing limits
An SM supports 2048 threads, 65,536 registers, and 96 KiB shared memory. A 256-thread block uses 48 registers/thread and 24 KiB shared memory. Limits are 8 blocks by threads, floor(65536/(256x48))=5 by registers, and 4 by shared memory. Thus 4 blocks reside, giving 1024/2048=50% occupancy.

TRY IT NOW - Occupancy bottleneck
An SM has 64 KiB shared memory. A block uses 20 KiB. Ignoring all other limits, how many blocks fit? Would reducing use to 16 KiB necessarily speed up the kernel?

SOLUTION
floor(64/20)=3 blocks fit. At 16 KiB, four blocks fit. Performance may improve through more latency-hiding warps, but it is not guaranteed: another resource may become limiting, and the optimization could add instructions or reduce locality.

6. OpenMP, MPI, and Hybrid Programming

6.1 Choose a model that matches the memory system

PROGRAMMING MODELS AND PROGRAMMER RESPONSIBILITY

How to read: The API changes what is implicit versus explicit, not the fundamental costs.

Model	Execution unit	Data organization	Programmer expresses
Pthreads	CPU threads	Shared address space	Thread creation, work division, synchronization
OpenMP	Threads and tasks	Shared process data	Parallel regions, worksharing, data attributes
CUDA/HIP	GPU threads and blocks	Host/device memory spaces	Thread mapping and device data
MPI	Processes/ranks	Process-local memory	Data distribution and messages
MPI+X	Ranks plus threads/GPU	Distributed between nodes; shared within node	Hierarchical parallelism and affinity

6.2 OpenMP fork-join and data sharing

OpenMP begins with one thread, forks a team for a parallel region, distributes work, and later joins. In a parallel for, the loop iteration variable is private. Arrays and loop bounds defined outside are usually shared. Variables declared inside an iteration are private to that iteration. default(none) is valuable because it forces every external variable to be classified deliberately.

```
double sum = 0.0;
#pragma omp parallel for default(none) shared(a,n) reduction(+:sum)
for (int i=0; i<n; ++i)
    sum += a[i];
```

Without reduction, sum has a race: several threads execute a read-modify-write on the same location. A reduction gives each thread a private partial sum and combines the partials. atomic or critical can also preserve correctness, but usually serializes more of the loop.

TRY IT NOW - Choose a synchronization primitive
Choose reduction, atomic, critical, barrier, or none for: scalar sum; one counter increment; a multi-statement shared data-structure update; a phase boundary; independent writes y[i].

SOLUTION
Use reduction for the sum, atomic for a supported single counter update, critical or a lock for the compound update, a barrier for the phase boundary, and no synchronization for y[i] when each iteration owns a distinct i.

6.3 Scheduling and tasks

schedule(static) has low overhead and good predictability. schedule(dynamic,chunk) repairs irregular load balance by assigning chunks at runtime. schedule(guided) begins with larger chunks and shrinks them, often balancing overhead and irregularity. Tasks are better when work forms an irregular dependency graph rather than a simple loop.

```
#pragma omp task depend(out:A)          // produce A
#pragma omp task depend(in:A) depend(out:B)  // use A -> B
#pragma omp task depend(in:A) depend(out:C)  // use A -> C
#pragma omp task depend(in:B,C)               // join before D
```

TRY IT NOW - Read the task graph
Which tasks above can execute concurrently?

SOLUTION
A must finish first. B and C then become ready and may execute concurrently. The final task waits until both B and C are complete. The graph is A -> {B,C} -> D.

6.4 MPI collectives and row-partitioned matvec

MPI ranks have separate address spaces. Communication is explicit, which makes data ownership visible. In row-partitioned y=Ax, each rank owns some rows of A and therefore produces the corresponding segment of y. Every rank needs x, so x must be broadcast or all-gathered before local multiplication.

```
MPI_Bcast(x, n, MPI_DOUBLE, 0, comm);
local_matvec(local_A, x, local_y);
MPI_Gather(local_y, local_m, MPI_DOUBLE,
           y, local_m, MPI_DOUBLE, 0, comm);
```

TRY IT NOW - All-gather meaning
Four ranks each own 256 floats. After MPI_Allgather, how many floats are in each receive buffer? How is this different from Gather?

SOLUTION
Each rank receives four segments, so every receive buffer has 4x256=1024 floats. Gather places the 1024-float concatenation only at the root; Allgather replicates it at all four ranks.

6.5 Blocking-message deadlock and hybrid design

If every rank in a ring performs a blocking send to the next rank before posting a receive from the previous rank, all ranks may wait in a circular dependency. Small messages can appear to work because of eager buffering, but correctness must not depend on that implementation detail. Use MPI_Sendrecv or post MPI_Irecv before MPI_Isend and complete both with MPI_Waitall.

TRY IT NOW - MPI+OpenMP design
Give one reason to use MPI between nodes and OpenMP within a node for matrix-vector multiplication.

SOLUTION
MPI matches distributed memory between nodes, while OpenMP lets threads share the rank's local matrix and vector within a node. This can reduce the number of MPI ranks, duplicated data, and messages, but requires attention to NUMA placement and thread affinity.

7. Exam Connections and Final Review

7.1 How the practice-midterm questions map to concepts

PRACTICE MIDTERM MAP - WHERE EACH ANSWER NOW LIVES

How to read: Every computational problem and explanation appears next to its concept in this guide.

Practice item	Concept location	Answer idea
Q1 True/False	Chapters 1, 3, 4, 6	Judge the mechanism and its scope, not a memorized slogan
Q2 definitions	Sections 3.4, 4.5, 6.4	SMT, All-gather, texture/read-only cache
Q3 Amdahl	Section 1.2	S=1/[s+(1-s)/p], limit=1/s
Q4 and Q6 indexing	Section 5.1	Two contiguous block-sized sections
Q5 transpose race	Section 5.3	One-warp course case; barrier for general case
Q7 GEMM requests	Section 5.4	N naive and N/T tiled
Q8 shared instances	Section 4.2	One logical shared instance per block
Q9 shared vs. registers	Section 4.5	Cross-thread sharing and indexed exchange

7.2 Formula sheet with interpretation

FORMULAS TO EXPLAIN, NOT JUST MEMORIZE

How to read: The last column is the sentence you should be able to say aloud.

Topic	Formula	Interpretation
Speedup	$S=T_1/T_p$	Parallel time relative to the serial baseline
Efficiency	E=S/p	Fraction of ideal p-fold speedup
Amdahl	S=1/[s+(1-s)/p]	Fixed-size time is serial remainder plus divided work
Gustafson	S=p-(p-1)s	Scaled parallel work grows with p at fixed time
Roofline	$P<=\min(P_{peak}, BI)$	Lower compute or bandwidth ceiling wins
CUDA 1D	i=blockIdx.x blockDim.x+threadIdx.x	Earlier blocks plus local offset
GEMM requests	N naive; N/T tiled	A staged load serves T consumers
Occupancy	resident threads / maximum threads	Residency is limited by the tightest resource

7.3 Common reasoning mistakes

ERROR PATTERNS AND THEIR REPAIRS

How to read: Use this as a final self-check before submitting an answer.

Mistake	Why it fails	Repair
Treating processor count as speedup	Overheads and serial work remain	Compute T_1/T_p
Calling every branch divergent	Divergence is within a warp	Check whether lanes choose different paths
Checking adjacency within one thread	Transactions group lanes per instruction	List addresses across the warp
Using __syncthreads for blocks	It is block-local	Split kernels or use supported cooperative sync
Assuming shared memory is always faster	Reuse may not repay barriers/conflicts	Count eliminated global requests
Maximizing occupancy blindly	Other bottlenecks may dominate	Profile transactions, divergence, and throughput
Using a shared OpenMP accumulator	Read-modify-write races	Use reduction or a suitable primitive

Final five-question self-test

Can you derive Amdahl from normalized time? Can you derive the Roofline bandwidth term from bytes/second times FLOPs/byte? Can you list one warp's addresses? Can you name every producer-consumer dependency before placing a barrier? Can you state who owns each piece of data before choosing an MPI collective?

8. References

Primary course sources

- Intro to Parallel Computing 1-3: motivation, metrics, algorithm design, programming models, OpenMP, MPI, and CUDA.
- CMP and Vector Computers: chip multiprocessors, superscalar execution, SMT, SIMD/vector processing, coherence, and consistency.
- GPU Architectures 1-2: throughput design, SIMT, NVIDIA/AMD/Arm execution, divergence, and GPU memory systems.
- CUDA basics_extended: thread mapping, memory spaces, coalescing, shared-memory tiling, synchronization, and occupancy.
- intro_to_HPC_OpenMP and omp-hands-on-SC08: fork-join, data sharing, reduction, scheduling, false sharing, and tasks.
- Computing2 Practice Midterm, Fall 2026: exam style and required reasoning patterns.

Further technical references

- NVIDIA CUDA Programming Guide and CUDA C++ Best Practices Guide.
- OpenMP API Specification 6.0.
- MPI 4.1 Standard.
- Gene Amdahl, 1967, Validity of the Single Processor Approach to Achieving Large Scale Computing Capabilities.
- John Gustafson, 1988, Reevaluating Amdahl's Law.
- Williams, Waterman, and Patterson, Roofline: An Insightful Visual Performance Model.

Architecture-dependent details
Warp/wavefront width, bank count, maximum threads per block, register capacity, and transaction behavior vary by generation. For exam questions, use the stated assumptions and the lecture's model. For real devices, verify the programming guide and device properties.